

Fintech Ledger / Wallet API

Project overview · learning-through-building backend project in NestJS

What This Project Is

A backend for a digital wallet system — the kind of thing behind apps like PalmPay, Opay, or a savings app. Users hold balances. They send money to each other, deposit, and withdraw. The core job of the system is to make sure the numbers are always correct: no balance goes negative by mistake, no one gets charged twice, and if the server crashes mid-transfer, money does not vanish or duplicate.

The design is based on **double-entry bookkeeping** — the same method real banks and accounting systems use. Instead of storing a balance as a single number you update directly, every transaction creates two linked records: money leaves one account and enters another, and the two sides must always balance to zero. This makes every movement of money traceable and auditable.

Why This Project

Money bugs are unforgiving. A to-do app can have a bug and nobody notices. A wallet app with a bug means someone's balance is wrong — so it forces real thinking about correctness, not just getting the happy path to work. It pushes into concurrency, data integrity, and system design in a way most CRUD projects don't, while staying scoped enough to actually finish.

Where This Places You

Finishing this project gets you to a solid mid-to-senior level as a single-service backend engineer — deep NestJS architecture fluency, real database correctness thinking, and testing discipline beyond the happy path. This covers what most backend interviews and remote job specs test for.

It does **not** cover (deliberately out of scope for this project):

- Microservices — multiple services talking via a message broker (Kafka / RabbitMQ)
 - Distributed locking across multiple app servers (Redis Redlock and similar)
 - Observability at scale — distributed tracing, metrics dashboards (Prometheus / Grafana)
 - GraphQL
 - Infra/deployment beyond Docker — Kubernetes, large-scale CI/CD pipelines
-

Technical Areas Covered

Eleven areas, meant to be tackled roughly in order as the project is built out.

1. Data Modeling

Accounts, transactions, and ledger entries as separate tables. Balances are derived from ledger entries rather than stored as a single mutable field (or a hybrid: a cached balance kept in sync through

reconciliation).

2. Database Transactions & Atomicity

Using Prisma's transaction API so a transfer either fully succeeds or fully fails — never half-applied.

3. Concurrency Control

Row-level locking (SELECT ... FOR UPDATE) or optimistic locking with version numbers, to stop two simultaneous requests from double-spending the same balance.

4. Idempotency

Idempotency keys so that if a client retries a request — bad network, timeout — the transfer does not get applied twice.

5. Auth & Guards

JWT authentication, role-based access (user vs admin), and ownership checks so a user can only transact on their own account.

6. Validation

DTOs with class-validator: no negative amounts, currency checks, insufficient-balance checks.

7. Webhooks

Simulating a payment provider (Paystack / Flutterwave-style) calling back to confirm a deposit, including signature verification. Optional but valuable.

8. Audit Logging

Every balance-changing action recorded immutably and queryable by an admin.

9. Background Jobs

BullMQ for things like generating monthly statements or processing scheduled transfers.

10. Testing

The part that proves the rest works — writing tests that simulate concurrent requests hitting the same account, to confirm the locking strategy actually holds.

11. Error Handling & Exception Filters

Custom exceptions (InsufficientFundsException, AccountFrozenException) mapped to proper HTTP responses via Nest's exception filters.

Rough Timeline

Estimated at roughly 1-2 hours on weekdays, more on weekends, alongside the Jasper placement and job search. Total: approximately 5-7 weeks at that pace.

Phase	Focus	Estimate
-------	-------	----------

1	Schema, CRUD, DTOs, Swagger	4-6 days
2	Auth, guards, RBAC, account ownership	4-6 days
3	Ledger logic, transactions, atomicity	5-7 days
4	Concurrency (locking) + idempotency	5-8 days
5	Webhook simulation + signature verification	2-3 days
6	Audit logging	2 days
7	Background jobs (BullMQ)	3-4 days
8	Testing, including concurrency tests	5-7 days
9	Docker, CI, deploy	2-3 days

Note: Phase 4 (concurrency) is typically the hardest phase — budget extra time there. Phase 8 (testing) is where the concurrency work actually gets proven; it's worth not compressing.

Notes

This document covers the project scope only. The PRD — detailed requirements, endpoint list, and schema design — comes next.